

Faculty Event Management System

Complete Developer Guide — Head to Tail

Spring Boot • Thymeleaf • MySQL • Maven

Version	1.0
Audience	Backend & Frontend Developers
Stack	Java 17 · Spring Boot 3.x · Thymeleaf · MySQL 8
Purpose	Venue Booking & Conflict Prevention for Faculty Societies

TABLE OF CONTENTS

- 1. Project Overview & Core Requirements 3
- 2. System Architecture 4
- 3. Technology Stack & Tools 4
- 4. Database Design — MySQL Tables 5
- 5. Maven Project Setup (pom.xml) 8
- 6. Application Configuration 9
- 7. Entity Classes (JPA Models) 9
- 8. Repository Layer 12
- 9. Service Layer 12
- 10. Controller Layer 14
- 11. Thymeleaf Templates 17
- 12. Security — Spring Security 20
- 13. Conflict Detection Logic 21
- 14. Email Notifications 22
- 15. Main Functionalities Summary 22
- 16. Folder Structure 23
- 17. Testing 24
- 18. Running the Application 24
- 19. Future Enhancements 25

1. PROJECT OVERVIEW & CORE REQUIREMENTS

1.1 Problem Statement

Multiple student societies within a faculty often try to book the same limited venues (Auditorium, Seminar Hall, Lab) for their events and sessions on overlapping dates and times. Without a central system, double-bookings cause conflicts, wasted resources, and organisational chaos. This system solves that problem by providing a centralised, role-based venue booking platform with automatic conflict detection.

1.2 Functional Requirements

ID	Requirement	Description
FR-01	User Registration & Login	Students, Society Admins, and Faculty Admins can register and log in securely.
FR-02	Society Management	Create, update, and deactivate societies; assign society admins.
FR-03	Venue Management	CRUD operations for venues including capacity, facilities, and availability status.
FR-04	Event Creation	Societies can create events with name, date, time, expected headcount, and requirements.
FR-05	Venue Booking Request	Society submits a booking request for a specific venue, date, and time slot.
FR-06	Conflict Detection	System automatically detects overlapping bookings for the same venue and time.
FR-07	Booking Approval Workflow	Faculty Admin reviews and approves/rejects pending booking requests.
FR-08	Dashboard	Role-specific dashboards: admin sees all, society sees own events.
FR-09	Booking Calendar View	Visual calendar showing venue availability across dates.
FR-10	Email Notifications	Automated emails on booking confirmation, rejection, and reminder.
FR-11	Booking History	Complete audit log of all bookings per venue and per society.
FR-12	Reports	Admin can export bookings as PDF/CSV reports.

1.3 Non-Functional Requirements

- Security — Role-based access control; passwords hashed with BCrypt.
- Scalability — Stateless REST-friendly architecture; pagination on list pages.
- Usability — Responsive Thymeleaf UI using Bootstrap 5; mobile-friendly.
- Reliability — Transactional bookings; rollback on partial failure.
- Auditability — Created-at / updated-at timestamps on all records.
- Performance — Indexed foreign keys; lazy loading on associations.

2. SYSTEM ARCHITECTURE

The application follows a classic MVC (Model-View-Controller) layered architecture with Spring Boot as the backbone. The layers are:

Layer	Technology	Responsibility
Presentation	Thymeleaf + Bootstrap 5	Render HTML pages; form binding; validation display
Controller	Spring MVC (@Controller)	Handle HTTP requests; call service layer; return views
Service	Spring @Service	Business logic; conflict detection; email triggers
Repository	Spring Data JPA	Database CRUD; custom JPQL queries
Domain / Model	JPA @Entity classes	Map Java objects to MySQL tables
Security	Spring Security	Authentication; role-based URL protection
Database	MySQL 8	Persist all application data

■ TIP

Data flows: Browser → Controller → Service → Repository → MySQL, and back via Thymeleaf template rendering.

3. TECHNOLOGY STACK & TOOLS

Component	Technology	Version
Language	Java	17 (LTS)
Framework	Spring Boot	3.2.x
View Engine	Thymeleaf	3.x (bundled)
ORM	Spring Data JPA + Hibernate	6.x
Database	MySQL	8.0+
Security	Spring Security	6.x
Build Tool	Maven	3.9+
CSS Framework	Bootstrap	5.3
Icons	Bootstrap Icons / Font Awesome	5.x
Email	Spring Mail + Thymeleaf Email Templates	—
Validation	Jakarta Bean Validation (Hibernate Validator)	8.x
IDE	IntelliJ IDEA / Eclipse / VS Code	Any
DB Client	MySQL Workbench / DBeaver	Any

4. DATABASE DESIGN — MySQL TABLES

The schema uses 7 core tables with clear foreign-key relationships to enforce referential integrity and avoid orphan records.

TABLE: users

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT, NOT NULL	Unique user identifier
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
email	VARCHAR(100)	UNIQUE, NOT NULL	User email address
password	VARCHAR(255)	NOT NULL	BCrypt hashed password
full_name	VARCHAR(100)	NOT NULL	Display name
role	ENUM(ADMIN, SOCIETY_ADMIN, MEMBER, NONMEMBER)	NOT NULL, DEFAULT MEMBER	User role
enabled	BOOLEAN	NOT NULL, DEFAULT TRUE	Account active flag
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Registration timestamp
updated_at	TIMESTAMP	ON UPDATE NOW()	Last update timestamp

■ TIP

Passwords must NEVER be stored as plain text. Always encode with BCryptPasswordEncoder.

TABLE: societies

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique society ID
name	VARCHAR(100)	UNIQUE, NOT NULL	Society name
description	TEXT	NULLABLE	Society description
contact_email	VARCHAR(100)	NOT NULL	Public contact email
logo_url	VARCHAR(255)	NULLABLE	Path to logo image
admin_id	BIGINT	FK → users(id), NOT NULL	Society admin user
active	BOOLEAN	NOT NULL, DEFAULT TRUE	Active status
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Creation timestamp

■ TIP

Each society has exactly one admin. The admin can be changed by the Faculty Admin.

TABLE: venues

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique venue ID
name	VARCHAR(100)	UNIQUE, NOT NULL	Venue name (e.g. Auditorium)
location	VARCHAR(150)	NOT NULL	Building / floor / room
capacity	INT	NOT NULL	Maximum seating capacity
has_projector	BOOLEAN	DEFAULT FALSE	Projector available?

has_ac	BOOLEAN	DEFAULT FALSE	AC available?
has_sound	BOOLEAN	DEFAULT FALSE	Sound system available?
description	TEXT	NULLABLE	Additional details
active	BOOLEAN	DEFAULT TRUE	Venue bookable flag
created_at	TIMESTAMP	DEFAULT NOW()	Creation timestamp

■ TIP
Add INDEX on active column to speed up availability queries.

TABLE: events

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique event ID
title	VARCHAR(150)	NOT NULL	Event title
description	TEXT	NULLABLE	Event description
event_type	ENUM(WORKSHOP, SEMINAR, COMPETITION, CULTURAL, OTHER)	NOT NULL	Category of event
society_id	BIGINT	FK → societies(id), NOT NULL	Organising society
organiser_id	BIGINT	FK → users(id), NOT NULL	User who created event
expected_attendees	INT	NOT NULL	Expected headcount
status	ENUM(DRAFT, PUBLISHED, CANCELLED, COMPLETED)	NOT NULL	Event lifecycle status
created_at	TIMESTAMP	DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMP	ON UPDATE NOW()	Last update

■ TIP

An event can have multiple sessions (different dates/venues). See venue_bookings table.

TABLE: venue_bookings ★ (Conflict Prevention Key Table)

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique booking record
event_id	BIGINT	FK → events(id), NOT NULL	Linked event
venue_id	BIGINT	FK → venues(id), NOT NULL	Requested venue
society_id	BIGINT	FK → societies(id), NOT NULL	Requesting society
requested_by	BIGINT	FK → users(id), NOT NULL	User who made request
booking_date	DATE	NOT NULL	Date of the booking
start_time	TIME	NOT NULL	Session start time
end_time	TIME	NOT NULL	Session end time
status	ENUM(PENDING, APPROVED, REJECTED, CANCELLED)	NOT NULL	Approval status
admin_note	TEXT	NULLABLE	Rejection/approval note
reviewed_by	BIGINT	FK → users(id), NULLABLE	Admin who reviewed
reviewed_at	TIMESTAMP	NULLABLE	Review timestamp
created_at	TIMESTAMP	DEFAULT NOW()	Request created

■ TIP

UNIQUE CONSTRAINT or query guard on (venue_id, booking_date, start_time, end_time) prevents double-booking. Conflict query checks: same venue_id AND booking_date AND status=APPROVED AND time ranges overlap.

TABLE: notifications

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Notification ID
user_id	BIGINT	FK → users(id), NOT NULL	Recipient user

message	TEXT	NOT NULL	Notification text
type	ENUM(BOOKING_APPROVED, BOOKING_REJECTED, REMINDER, GENERAL)	NOT NULL	Notification type
is_read	BOOLEAN	DEFAULT FALSE	Read flag
created_at	TIMESTAMP	DEFAULT NOW()	Sent timestamp

■ **TIP**
Poll unread notifications on dashboard load. Mark as read when user views them.

TABLE: society_members

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Row ID
society_id	BIGINT	FK → societies(id), NOT NULL	Society
user_id	BIGINT	FK → users(id), NOT NULL	Member user
joined_at	TIMESTAMP	DEFAULT NOW()	Join date
role_in_society	ENUM(ADMIN, MEMBER)	NOT NULL, DEFAULT MEMBER	Role within society

■ **TIP**
Composite UNIQUE KEY on (society_id, user_id) prevents duplicate memberships.

4.8 Entity Relationship Summary

users (1) ■■■■■ (many) society_members (many) ■■■■■ (1) societies | societies (1) ■■■■■ (many) events (1) ■■■■■ (many) venue_bookings (many) ■■■■■ (1) venues | users (1) ■■■■■ (many) venue_bookings | users (1) ■■■■■ (many) notifications

```
-- CONFLICT DETECTION SQL (used in Service layer)
```

```
SELECT COUNT(*) FROM venue_bookings
```

```
WHERE venue_id = :venueId
```

```
    AND booking_date = :date
```

```
    AND status = 'APPROVED'
```

```
    AND start_time < :endTime
```

```
    AND end_time   > :startTime;
```

```
-- Returns > 0 means CONFLICT exists
```

5. MAVEN PROJECT SETUP — pom.xml

Generate the project at <https://start.spring.io> with the following dependencies, then add the extras below.

```
<dependencies>
  <!-- Spring Web (MVC + Embedded Tomcat) -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId></dependency>
  <!-- Thymeleaf -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId></dependency>
  <!-- Thymeleaf + Spring Security integration -->
  <dependency><groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId></dependency>
  <!-- Spring Data JPA -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <!-- MySQL Connector -->
  <dependency><groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId><scope>runtime</scope></dependency>
  <!-- Spring Security -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId></dependency>
  <!-- Validation -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId></dependency>
  <!-- Email -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId></dependency>
  <!-- Lombok (optional, reduces boilerplate) -->
  <dependency><groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId><optional>true</optional></dependency>
</dependencies>
```

6. APPLICATION CONFIGURATION

src/main/resources/application.properties

```
# Database
spring.datasource.url=jdbc:mysql://localhost:3306/faculty_events?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=your_password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.properties.hibernate.format_sql=true

# Thymeleaf
spring.thymeleaf.cache=false

# Mail (Gmail SMTP example)
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=your@gmail.com
spring.mail.password=app_password_here
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

# App
server.port=8080
app.name=Faculty Event Manager
```

NOTE

Never commit real passwords to Git. Use environment variables or application-secret.properties (gitignored).

7. ENTITY CLASSES (JPA MODELS)

7.1 User.java

```
@Entity @Table(name = "users")
@Data @NoArgsConstructor @AllArgsConstructor
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique=true, nullable=false, length=50)
    private String username;

    @Column(unique=true, nullable=false, length=100)
    @Email private String email;

    @Column(nullable=false)
    private String password; // BCrypt hashed

    @Column(name="full_name", nullable=false)
    private String fullName;

    @Enumerated(EnumType.STRING)
    @Column(nullable=false)
    private Role role = Role.MEMBER; // ADMIN | SOCIETY_ADMIN | MEMBER

    private boolean enabled = true;

    @CreationTimestamp
    @Column(name="created_at", updatable=false)
    private LocalDateTime createdAt;
}
```

7.2 Venue.java

```
@Entity @Table(name = "venues")
@Data @NoArgsConstructor
public class Venue {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique=true, nullable=false) private String name;
    @Column(nullable=false) private String location;
    @Column(nullable=false) private int capacity;

    private boolean hasProjector;
    private boolean hasAc;
    private boolean hasSound;
    @Column(columnDefinition="TEXT") private String description;
    private boolean active = true;

    @CreationTimestamp private LocalDateTime createdAt;
}
```

7.3 VenueBooking.java — the conflict-prevention entity

```
@Entity @Table(name = "venue_bookings")
@Data @NoArgsConstructor
public class VenueBooking {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
```

```
@JoinColumn(name="event_id", nullable=false) private Event event;
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name="venue_id", nullable=false) private Venue venue;
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name="society_id",nullable=false) private Society society;
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name="requested_by",nullable=false) private User requestedBy;
@Column(nullable=false) private LocalDate bookingDate;
@Column(nullable=false) private LocalTime startTime;
@Column(nullable=false) private LocalTime endTime;
@Enumerated(EnumType.STRING)
@Column(nullable=false)
private BookingStatus status = BookingStatus.PENDING;
// PENDING | APPROVED | REJECTED | CANCELLED
private String adminNote;
@ManyToOne(fetch=FetchType.LAZY)
@JoinColumn(name="reviewed_by") private User reviewedBy;
private LocalDateTime reviewedAt;
@CreationTimestamp private LocalDateTime createdAt;
}
```

8. REPOSITORY LAYER

VenueBookingRepository.java — custom conflict query

```
@Repository
public interface VenueBookingRepository extends JpaRepository<VenueBooking, Long> {

    // Conflict detection: same venue, same date, APPROVED, overlapping time
    @Query("""
        SELECT COUNT(b) FROM VenueBooking b
        WHERE b.venue.id = :venueId
            AND b.bookingDate = :date
            AND b.status = 'APPROVED'
            AND b.startTime < :endTime
            AND b.endTime  > :startTime
        """)
    long countConflicts(@Param("venueId") Long venueId,
                        @Param("date")    LocalDate date,
                        @Param("startTime") LocalTime startTime,
                        @Param("endTime")  LocalTime endTime);

    // Find all bookings for a venue on a date (for calendar view)
    List<VenueBooking> findByVenueIdAndBookingDate(Long venueId, LocalDate date);

    // All bookings by a society
    Page<VenueBooking> findBySocietyId(Long societyId, Pageable pageable);

    // Pending bookings for admin
    List<VenueBooking> findByStatusOrderByCreatedAtAsc(BookingStatus status);
}
```

■ TIP

Also create UserRepository (findByUsername, findByEmail), VenueRepository, EventRepository, SocietyRepository — all extend JpaRepository.

9. SERVICE LAYER

9.1 VenueBookingService.java

```
@Service
@Transactional
public class VenueBookingService {

    @Autowired private VenueBookingRepository bookingRepo;
    @Autowired private NotificationService notifyService;
    @Autowired private EmailService emailService;

    public VenueBooking requestBooking(VenueBookingRequest dto, User requestedBy) {
        // 1. Check conflict
        long conflicts = bookingRepo.countConflicts(
            dto.getVenueId(), dto.getDate(),
            dto.getStartTime(), dto.getEndTime());
        if (conflicts > 0) {
            throw new BookingConflictException(
                "Venue is already booked for this time slot!");
        }
        // 2. Validate end > start
        if (!dto.getEndTime().isAfter(dto.getStartTime()))
            throw new IllegalArgumentException("End time must be after start time");
        // 3. Persist booking
        VenueBooking booking = mapDtoToEntity(dto, requestedBy);
        booking = bookingRepo.save(booking);
        // 4. Notify admin
        notifyService.notifyAdmins("New booking request from " + requestedBy.getFullName());
        return booking;
    }

    public VenueBooking approveBooking(Long bookingId, User admin, String note) {
        VenueBooking b = bookingRepo.findById(bookingId)
            .orElseThrow(() -> new ResourceNotFoundException("Booking not found"));
        // Re-check conflict in case another was approved meanwhile
        long conflicts = bookingRepo.countConflicts(
            b.getVenue().getId(), b.getBookingDate(),
            b.getStartTime(), b.getEndTime());
        if (conflicts > 0)
            throw new BookingConflictException("Conflict found at approval time!");
        b.setStatus(BookingStatus.APPROVED);
        b.setReviewedBy(admin);
        b.setReviewedAt(LocalDateDateTime.now());
        b.setAdminNote(note);
        bookingRepo.save(b);
        emailService.sendApprovalEmail(b);
        return b;
    }

    public VenueBooking rejectBooking(Long bookingId, User admin, String reason) {
        VenueBooking b = bookingRepo.findById(bookingId)
            .orElseThrow(() -> new ResourceNotFoundException("Booking not found"));
        b.setStatus(BookingStatus.REJECTED);
    }
}
```



```
b.setReviewedBy(admin);  
b.setReviewedAt(LocalDateTime.now());  
b.setAdminNote(reason);  
bookingRepo.save(b);  
emailService.sendRejectionEmail(b, reason);  
return b;  
}  
}
```

10. CONTROLLER LAYER

10.1 AuthController.java

```
@Controller
public class AuthController {
    @Autowired UserService userService;

    @GetMapping("/login")
    public String loginPage() { return "auth/login"; }

    @GetMapping("/register")
    public String registerForm(Model model) {
        model.addAttribute("user", new UserRegistrationDto());
        return "auth/register";
    }

    @PostMapping("/register")
    public String register(@Valid @ModelAttribute("user") UserRegistrationDto dto,
        BindingResult result, RedirectAttributes ra) {
        if (result.hasErrors()) return "auth/register";
        userService.registerUser(dto);
        ra.addFlashAttribute("success", "Account created! Please login.");
        return "redirect:/login";
    }
}
```

10.2 BookingController.java

```
@Controller
@RequestMapping("/bookings")
@PreAuthorize("hasAnyRole('ADMIN', 'SOCIETY_ADMIN')")
public class BookingController {
    @Autowired VenueBookingService bookingService;
    @Autowired VenueService venueService;
    @Autowired EventService eventService;

    @GetMapping("/new")
    public String newBookingForm(Model model) {
        model.addAttribute("booking", new VenueBookingDto());
        model.addAttribute("venues", venueService.getActiveVenues());
        model.addAttribute("events", eventService.getCurrentUserEvents());
        return "bookings/form";
    }

    @PostMapping("/new")
    public String submitBooking(@Valid @ModelAttribute("booking") VenueBookingDto dto,
        BindingResult result,
        @AuthenticationPrincipal UserDetails ud,
        RedirectAttributes ra, Model model) {
        if (result.hasErrors()) {
            model.addAttribute("venues", venueService.getActiveVenues());
            return "bookings/form";
        }
        try {
            User user = userService.findByUsername(ud.getUsername());
        }
    }
}
```

```

        bookingService.requestBooking(dto, user);
        ra.addFlashAttribute("success", "Booking request submitted!");
    } catch (BookingConflictException e) {
        model.addAttribute("error", e.getMessage());
        model.addAttribute("venues", venueService.getActiveVenues());
        return "bookings/form";
    }
    return "redirect:/bookings/my";
}

@GetMapping("/my")
public String myBookings(Model model,
                        @AuthenticationPrincipal UserDetails ud,
                        Pageable pageable) {
    User user = userService.findByUsername(ud.getUsername());
    model.addAttribute("bookings",
        bookingService.getBookingsForUser(user, pageable));
    return "bookings/list";
}
}

```

10.3 AdminBookingController.java

```

@Controller
@RequestMapping("/admin/bookings")
@PreAuthorize("hasRole('ADMIN')")
public class AdminBookingController {

    @Autowired VenueBookingService bookingService;

    @GetMapping("/pending")
    public String pendingBookings(Model model) {
        model.addAttribute("bookings",
            bookingService.getPendingBookings());
        return "admin/bookings/pending";
    }

    @PostMapping("/{id}/approve")
    public String approve(@PathVariable Long id,
                        @RequestParam String note,
                        @AuthenticationPrincipal UserDetails ud,
                        RedirectAttributes ra) {
        User admin = userService.findByUsername(ud.getUsername());
        bookingService.approveBooking(id, admin, note);
        ra.addFlashAttribute("success", "Booking approved.");
        return "redirect:/admin/bookings/pending";
    }

    @PostMapping("/{id}/reject")
    public String reject(@PathVariable Long id,
                        @RequestParam String reason,
                        @AuthenticationPrincipal UserDetails ud,
                        RedirectAttributes ra) {
        User admin = userService.findByUsername(ud.getUsername());
        bookingService.rejectBooking(id, admin, reason);
        ra.addFlashAttribute("warning", "Booking rejected.");
        return "redirect:/admin/bookings/pending";
    }
}

```

```
}  
}
```

11. THYMELEAF TEMPLATES

11.1 base-layout.html (fragments/layout.html)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
  <meta charset="UTF-8">
  <title th:text="${pageTitle} + ' | EMS'">EMS</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
  <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap-icons/font/bootstrap-icons.css">
</head>
<body>
  <!-- Navbar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
    <div class="container">
      <a class="navbar-brand" th:href="@{/}">
        <i class="bi bi-calendar-event"></i> Faculty EMS
      </a>
      <div class="navbar-nav ms-auto">
        <a class="nav-link" th:href="@{/events}">Events</a>
        <a class="nav-link" th:href="@{/bookings/new}"
          sec:authorize="hasAnyRole('ADMIN','SOCIETY_ADMIN')">
          Book Venue</a>
        <a class="nav-link" th:href="@{/admin/bookings/pending}"
          sec:authorize="hasRole('ADMIN')">Pending</a>
        <form th:action="@{/logout}" method="post" class="d-inline">
          <button class="btn btn-outline-light btn-sm">Logout</button>
        </form>
      </div>
    </div>
  </nav>
  <div class="container mt-4" th:fragment="content"></div>
</body></html>
```

11.2 Booking Request Form (bookings/form.html) — key snippet

```
<div class="card shadow">
  <div class="card-header bg-primary text-white">Request a Venue Booking</div>
  <div class="card-body">
    <!-- Error alert -->
    <div th:if="${error}" class="alert alert-danger" th:text="${error}"></div>
    <form th:action="@{/bookings/new}" th:object="${booking}" method="post">
      <div class="mb-3">
        <label>Event</label>
        <select th:field="**{eventId}" class="form-select">
          <option th:each="e : ${events}"
            th:value="${e.id}" th:text="${e.title}"></option>
        </select>
      </div>
    </form>
  </div>
</div>
```

```
</div>
<div class="mb-3">
  <label>Venue</label>
  <select th:field="*{venueId}" class="form-select">
    <option th:each="v : ${venues}"
      th:value="${v.id}"
      th:text="${v.name} + ' (Cap: ' + ${v.capacity} + ')"></option>
  </select>
</div>
<div class="row">
  <div class="col"><label>Date</label>
    <input type="date" th:field="*{bookingDate}" class="form-control"></div>
  <div class="col"><label>Start Time</label>
    <input type="time" th:field="*{startTime}" class="form-control"></div>
  <div class="col"><label>End Time</label>
    <input type="time" th:field="*{endTime}" class="form-control"></div>
</div>
<button class="btn btn-success mt-3" type="submit">
  <i class="bi bi-send"></i> Submit Request
</button>
</form>
</div>
</div>
```

12. SECURITY — SPRING SECURITY

SecurityConfig.java

```
@Configuration @EnableWebSecurity @EnableMethodSecurity
public class SecurityConfig {

    @Autowired UserDetailsServiceImpl userDetailsService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/login", "/register", "/css/**", "/js/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/bookings/**").hasAnyRole("ADMIN", "SOCIETY_ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard", true)
                .failureUrl("/login?error")
            )
            .logout(logout -> logout
                .logoutSuccessUrl("/login?logout")
            );
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}
```

■ TIP

Use `@PreAuthorize("hasRole('ADMIN')")` on controller methods for fine-grained security instead of configuring everything in `filterChain`.

13. CONFLICT DETECTION LOGIC — DEEP DIVE

Time-range overlap detection uses the classic interval-overlap formula: two intervals $[A_start, A_end)$ and $[B_start, B_end)$ overlap if and only if $A_start < B_end$ AND $A_end > B_start$. The JPQL query in the repository directly implements this condition in the WHERE clause.

- Step 1: User fills booking form (venue, date, start_time, end_time).
- Step 2: Controller calls `bookingService.requestBooking(dto, user)`.
- Step 3: Service calls `bookingRepo.countConflicts(...)` with the JPQL overlap query.
- Step 4: If count > 0 → throw `BookingConflictException` → user sees error alert on form.
- Step 5: If count == 0 → save booking with status = PENDING.
- Step 6: Admin reviews pending bookings in `/admin/bookings/pending`.
- Step 7: On Admin APPROVE → service re-runs `countConflicts` to guard against race condition.
- Step 8: If still 0 conflicts → status = APPROVED, email sent to society admin.
- Step 9: Other PENDING requests for the same slot remain PENDING (admin rejects them).

NOTE

Double-check at approval time (step 7) is critical because two admins could be approving simultaneously. Use `@Transactional` with pessimistic locking on the approval method for high-traffic deployments.

Conflict Visualization Logic for Calendar View

```
// In CalendarController
@GetMapping("/calendar")
public String calendar(@RequestParam Long venueId,
                      @RequestParam @DateTimeFormat(iso=DateTimeFormat.ISO.DATE)
                      LocalDate month, Model model) {
    List<VenueBooking> bookings =
        bookingRepo.findByVenueIdAndMonthAndStatus(
            venueId, month.getYear(), month.getMonthValue(),
            BookingStatus.APPROVED);
    // Convert to calendar day-map for Thymeleaf
    Map<Integer, List<VenueBooking>> calMap = bookings.stream()
        .collect(Collectors.groupingBy(
            b -> b.getBookingDate().getDayOfMonth()));
    model.addAttribute("calMap", calMap);
    model.addAttribute("venue", venueService.findById(venueId));
    model.addAttribute("month", month);
    return "calendar/view";
}
```


14. EMAIL NOTIFICATIONS

```
@Service
public class EmailService {
    @Autowired JavaMailSender mailSender;
    @Value("${spring.mail.username}") String from;

    public void sendApprovalEmail(VenueBooking booking) {
        SimpleMailMessage msg = new SimpleMailMessage();
        msg.setTo(booking.getSociety().getContactEmail());
        msg.setFrom(from);
        msg.setSubject("[EMS] Booking Approved: " + booking.getEvent().getTitle());
        msg.setText(
            "Dear " + booking.getRequestedBy().getFullName() + ",\n\n" +
            "Your booking for " + booking.getVenue().getName() +
            " on " + booking.getBookingDate() +
            " (" + booking.getStartTime() + " - " + booking.getEndTime() + ")" +
            " has been APPROVED.\n\nFaculty EMS");
        mailSender.send(msg);
    }

    public void sendRejectionEmail(VenueBooking booking, String reason) {
        SimpleMailMessage msg = new SimpleMailMessage();
        msg.setTo(booking.getSociety().getContactEmail());
        msg.setSubject("[EMS] Booking Rejected");
        msg.setText("Reason: " + reason);
        mailSender.send(msg);
    }
}
```

15. MAIN FUNCTIONALITIES SUMMARY

Feature	Role	URL / Action
Register & Login	All	/register, /login
View all events	All	/events
Create Society	Admin	/admin/societies/new
Manage Venues	Admin	/admin/venues
Create Event	Society Admin	/events/new
Request Venue Booking	Society Admin	/bookings/new
Conflict Detection	System (auto)	On booking submit
Approve / Reject Booking	Admin	/admin/bookings/pending
My Bookings	Society Admin	/bookings/my
Calendar View	All (logged in)	/calendar?venueId=X&month=YYYY-MM
Dashboard	All	/dashboard (role-specific)
Email Notification	System (auto)	On approve/reject
Booking History	Admin	/admin/bookings/history
Member Management	Society Admin	/societies/{id}/members
Export Report	Admin	/admin/reports/export

16. FOLDER STRUCTURE

```

faculty-ems/
├── src/main/java/com/faculty/ems/
│   ├── EmsApplication.java
│   ├── config/
│   │   ├── SecurityConfig.java
│   │   ├── controller/
│   │   │   ├── AuthController.java
│   │   │   ├── DashboardController.java
│   │   │   ├── EventController.java
│   │   │   ├── BookingController.java
│   │   │   ├── VenueController.java
│   │   │   ├── SocietyController.java
│   │   │   ├── CalendarController.java
│   │   │   └── admin/
│   │   │       ├── AdminBookingController.java
│   │   │       └── AdminReportController.java
│   │   ├── model/                (entity classes)
│   │   │   ├── User.java
│   │   │   ├── Society.java
│   │   │   ├── Venue.java
│   │   │   ├── Event.java
│   │   │   ├── VenueBooking.java
│   │   │   ├── Notification.java
│   │   │   └── enums/ (Role, BookingStatus, EventType...)
│   │   ├── repository/          (JPA interfaces)
│   │   ├── service/             (business logic)
│   │   ├── dto/                 (form DTOs / request objects)
│   │   └── exception/           (custom exceptions)
│   ├── src/main/resources/
│   │   ├── application.properties
│   │   ├── static/
│   │   │   ├── css/custom.css
│   │   │   └── js/app.js
│   │   └── templates/
│   │       ├── fragments/layout.html
│   │       ├── auth/ (login.html, register.html)
│   │       ├── dashboard/ (dashboard.html)
│   │       ├── events/ (list.html, form.html, detail.html)
│   │       ├── bookings/ (form.html, list.html)
│   │       ├── venues/ (list.html, form.html)
│   │       ├── societies/ (list.html, form.html)
│   │       ├── calendar/ (view.html)
│   │       └── admin/ (bookings/pending.html, reports/...)
│   ├── src/test/java/
│   │   ├── ...service tests, controller tests
│   └── pom.xml

```

17. TESTING

Unit Test — Conflict Detection

```
@ExtendWith(MockitoExtension.class)
class VenueBookingServiceTest {

    @Mock VenueBookingRepository bookingRepo;
    @Mock EmailService emailService;
    @InjectMocks VenueBookingService bookingService;

    @Test
    void requestBooking_shouldThrowWhenConflictExists() {
        // Arrange
        when(bookingRepo.countConflicts(anyLong(), any(), any(), any()))
            .thenReturn(1L); // simulate conflict

        VenueBookingDto dto = new VenueBookingDto();
        dto.setVenueId(1L);
        dto.setDate(LocalDate.now().plusDays(3));
        dto.setStartTime(LocalTime.of(9,0));
        dto.setEndTime(LocalTime.of(11,0));

        // Act & Assert
        assertThrows(BookingConflictException.class,
            () -> bookingService.requestBooking(dto, new User()));
    }
}
```

18. RUNNING THE APPLICATION

- Step 1: Install Java 17+, Maven 3.9+, MySQL 8.
- Step 2: Create database: CREATE DATABASE faculty_events CHARACTER SET utf8mb4;
- Step 3: Update application.properties with your MySQL credentials.
- Step 4: Run: mvn spring-boot:run (or run EmsApplication.java from IDE).
- Step 5: Open browser: http://localhost:8080
- Step 6: First run: Spring will auto-create all tables (ddl-auto=update).
- Step 7: Register an account, then manually set role=ADMIN in DB for the first admin.
- Step 8: Log in as Admin and start creating Venues and Societies.

19. FUTURE ENHANCEMENTS

- REST API endpoints (for mobile app integration using Spring Boot + JWT).
- Google Calendar / Outlook integration for auto-adding approved bookings.
- QR code generation for event entry management.
- Recurring bookings support (e.g. weekly society meetings).
- Waitlist feature: auto-approve next pending request if an approved booking is cancelled.
- Docker & Docker Compose setup for containerized deployment.
- Actuator + Prometheus + Grafana for production monitoring.
- Advanced reporting: charts using Chart.js embedded in Thymeleaf.
- Multi-tenancy: extend to multiple faculties within one university.
- Push notifications via WebSocket (Spring WebSocket + STOMP).

End of Guide — Faculty Event Management System v1.0

Built with Spring Boot · Thymeleaf · MySQL · Bootstrap 5